

Minishell

Parser → Executor Interface Guide

Written by: **aleriaza** | For: **dprudnik** | Project: 42 Minishell

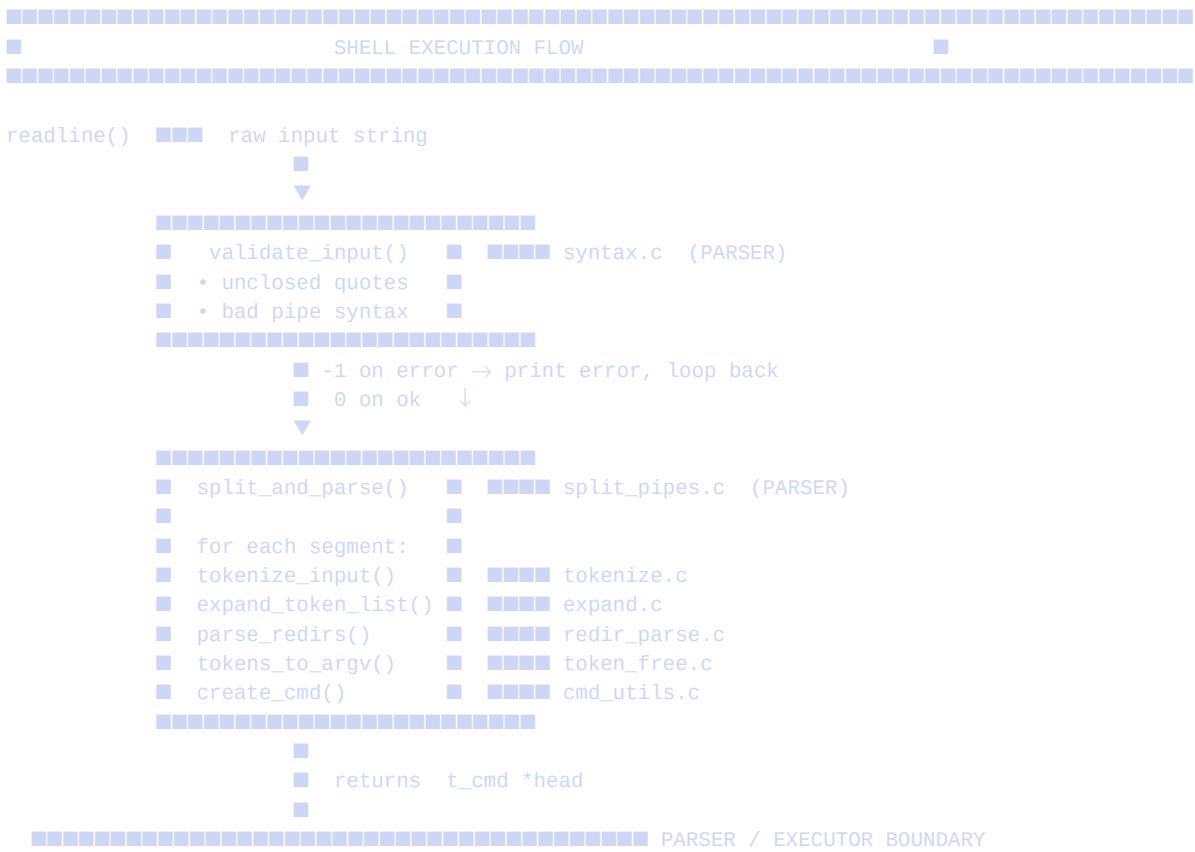
This document describes **exactly how the parser I wrote hands data to your executor**. It covers the structs, the functions you can call, the ones to avoid, and the full data flow from raw input to `execve()`. Read it once before you write a single line of executor code.

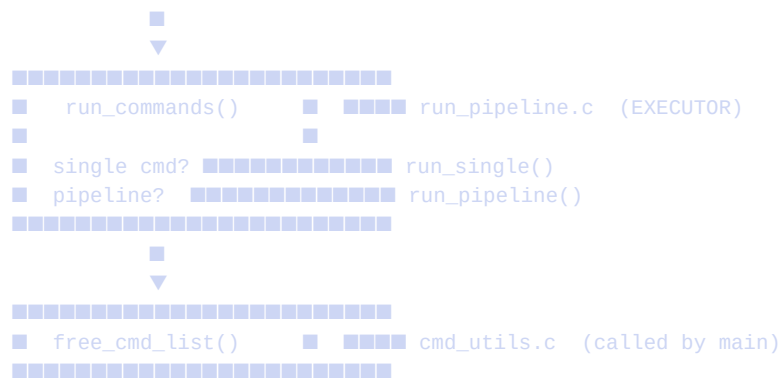
1. Overview — How the Two Parts Connect

I own everything up to and including the point where `split_and_parse()` returns. You own everything after that. The boundary is a single function call and a single struct:

```
// main.c (simplified)
cmds = split_and_parse(line, shell); // MY SIDE - returns ready-to-run list
if (!cmds)
    return ;
run_commands(cmds, shell);           // YOUR SIDE - executes the list
free_cmd_list(cmds);                 // MY SIDE - you call this after you're done
```

You receive a `t_cmd *` (a linked list). You run it. You never need to call any parser internals. Every string inside is already expanded and quote-free.





2. The Three Structs — Our Shared Language

These are the only data structures that cross the parser/executor boundary. I define them. You use them. Do not change their definitions.

```
t_shell (lives in main, passed everywhere by pointer)
=====
char **env      →  ["HOME=/home/...", "PATH=..", NULL]
int  exit_status →  0  (last command's exit code)
=====

t_cmd linked list (one node per pipe segment)
=====
args: ["ls", "-la",      args: ["grep",      args:
      "/tmp", NULL]      "foo", NULL]      ["wc", "-l",
redirs: NULL            redirs: [          NULL]
next: [                next: [            redirs: NULL
[                [                [                next: NULL
[                [                [                [
t_redir: [
{REDIR_OUT, "out.txt"}
→ NULL
```

2.1 t shell — Global shell state

- **env** — deep copy of the process environment. You mutate this with `env_set()` (export) and `builtin_unset()`. I read it **read-only** to expand \$VAR.
- **exit_status** — YOU write this after every command finishes. I read it to expand \$? in the next command. Set it to 0 on success, 1–255 on error, 127 for command not found, 128+N when a signal kills the child.

2.2 t redir — One redirection node

Apply the list **left-to-right** — last one wins, which is correct bash behaviour. The function `apply_redirs()` already does this. You just call it.

Function	File	What it does
split_and_parse()	split_pipes.c	MAIN ENTRY – split on , parse each segment, return t_cmd list
validate_input()	syntax.c	Check unclosed quotes and bad pipe syntax – call before parser
tokenize_input()	tokenize.c	Split raw string into t_token list, redirection ops as own tokens
expand_token_list()	expand.c	Expand \$VAR/\$? and strip quotes for every token in-place
parse_redirs()	redir_parse.c	Pull redir pairs from token list, build t_redir list
tokens_to_argv()	token_free.c	Convert token list to NULL-terminated char** argv
create_cmd()	cmd_utils.c	Alloc t_cmd; set args, redirs=NULL, next=NULL
free_cmd_list()	cmd_utils.c	Free entire t_cmd list after execution is done
cmd_count()	cmd_utils.c	Count nodes in t_cmd list (useful for allocating pid array)

4.2 Executor-side (already stubbed for you)

Function	File	What it does
run_commands()	run_pipeline.c	MAIN ENTRY – dispatch single or pipeline
apply_redirs()	redirections.c	Apply every redir left-to-right; return -1 on error
apply_one_redir()	redirections.c	Open file/heredoc, dup2 onto stdin or stdout
execute_child()	execute.c	Resolve path + execve; calls exit() on failure
find_command()	find_path.c	Search PATH dirs for cmd; return full path or NULL
is_builtin()	builtin_utils.c	Return 1 if cmd is echo/cd/pwd/env/export/unset/exit
execute_builtin()	builtin_utils.c	Dispatch t_cmd to the correct builtin function
env_set()	builtin_utils.c	Set or add KEY=VALUE in shell->env
get_env_var()	env_utils.c	Return pointer to value of a variable in env
print_error()	errors.c	Write 'minishell: <msg>' to stderr

5. Rules and Expectations

5.1 What you must do

- Implement `run_commands(t_cmd *cmds, t_shell *shell)` — this is your main entry point, called from `main.c`.
- Write `shell->exit_status` after every command completes. I read it for `$?` expansion.
- Call `apply_redirs(cmd->redirs)` inside each child process (or with `fd save/restore` in parent for builtins) before `exec`.
- Run builtins (`cd`, `export`, `unset`, `exit`) in the **parent process** so they affect the live shell state. All others in a forked child.
- For pipelines: fork one child per node, connect with `pipe()`, close all unused `fd` ends in parent, `waitpid()` all children. Set `exit_status` to the **last** child's code.

5.2 What you must NOT do

- ✗ Call `free_cmd_list()` before execution finishes — `main.c` does this after your function returns.
- ✗ Call any parser internals: `tokenize_input()`, `expand_token()`, `parse_redirs()`, `split_and_parse()`.

✗ Modify the `t_cmd` list during iteration (no re-ordering, no unlinking nodes).

✗ Change the struct definitions in `minishell.h`.

6. Minimal Executor Skeleton

This is the shape your executor should follow. Every detail of `apply_redirs`, `find_command`, and `execute_child` is already written — you connect the pieces.

```
// run_pipeline.c

static int run_builtin_with_redirs(t_cmd *cmd, t_shell *shell)
{
    int saved_in = dup(STDIN_FILENO);
    int saved_out = dup(STDOUT_FILENO);
    if (apply_redirs(cmd->redirs) == -1) { close(saved_in); close(saved_out); return 1;
}
    int ret = execute_builtin(cmd, shell);
    dup2(saved_in, STDIN_FILENO); close(saved_in);
    dup2(saved_out, STDOUT_FILENO); close(saved_out);
    return ret;
}

static int run_external(t_cmd *cmd, t_shell *shell)
{
    pid_t pid = fork();
    int status = 0;
    if (pid == 0)
    {
        if (apply_redirs(cmd->redirs) == -1) exit(1);
        execute_child(cmd->args, shell->env); // never returns on success
    }
    waitpid(pid, &status, 0);
    if (WIFEXITED(status)) return WEXITSTATUS(status);
    if (WIFSIGNALED(status)) return 128 + WTERMSIG(status);
    return 1;
}

static int run_single(t_cmd *cmd, t_shell *shell)
{
    if (!cmd->args || !cmd->args[0]) return 0;
    if (is_builtin(cmd->args[0])) return run_builtin_with_redirs(cmd, shell);
    return run_external(cmd, shell);
}

int run_commands(t_cmd *cmds, t_shell *shell)
{
    int ret = 0;
    if (!cmds) return 0;
    if (!cmds->next) // single command
    {
        ret = run_single(cmds, shell);
        shell->exit_status = ret;
        return ret;
    }
    ret = run_pipeline(cmds, shell); // pipeline (execute_pipes.c)
    shell->exit_status = ret;
    return ret;
}
```

7. Quick Reference Card

Parser output guarantees

- All strings in `args[]` and `redir->file` are **heap-allocated, fully expanded** (`$VAR`, `$?`), and **quote-stripped**.
- `args` is never `NULL`. `args[0]` can be `NULL` (redir-only segment — rare edge case).
- `redirs` is `NULL` when there are no redirections.
- `next` is `NULL` for the last command.
- The list is valid until `free_cmd_list()` is called.

exit_status contract

```
shell->exit_status = 0;           // success
shell->exit_status = 1;           // general error
shell->exit_status = 2;           // misuse of shell / syntax (set by parser)
shell->exit_status = 127;         // command not found
shell->exit_status = 128 + N;     // killed by signal N
shell->exit_status = prev_val;    // what $? expands to – YOU write this
```

Include path

```
// Your executor files use:
#include "minishell.h"          // if compiled with -I includes
// OR
#include "../../includes/minishell.h" // relative path from srcs/executor/
```